

Report 1: Project Progress

University/Department: California State University, Dominguez Hills

Course/Thesis/Project Title: CSC 599 Project

Semester/Year: Fall 2025

Project Title: A Hyper-Personalized Financial Product Engine Using Generative AI

Student Name: Sumaiya Shaikh

Student ID: 213028116

Instructor / Committee Chair: Dr. Benyamin Ahmadnia

Date: 10-27-2025

1. Abstract / Summary of Progress

This report is intended to share progress on the “Hyper-Personalized Financial Product Engine” project and to mark the completion of the “Foundation & Data” workstream (Weeks 1– 2). This deliverable represents the phase of the project moving from the descriptive design and analysis to a functional baseline implementation. The design for the triadic (3 tier) system architecture, which centers on a Flask backend and React frontend, has been successfully carried out. The database schema (User, Transaction, ProductCatalog) has been transformed into implemented code using Flask-SQLAlchemy. Additionally, basic user authentication (registering, logging in and password hashing) is fully functional. A synthetic dataset (1,000 users and 1,000,000 transactions) has been generated with the Faker library and populated in the SQLite database to set them up for modeling training. All foundational workstream goals have been completed for this iteration, the project is on schedule, and the project moves to model building (Weeks 3–6).

2. Introduction and Objectives (Recap)

This project seeks to address a key problem in the modern FinTech environment. In this section we present a summary of the project’s problem, motivation, goals, and key ideas as they relate to the problem area.

2.1. Problem Statement

The primary problem being certainly a breakdown in ability to not just communicate but also fulfill matching individuals with highly relevant and contextual financial products in a safe digital environment. The digitalization of banking has not clarified choices; in fact, it has confused consumers and resulted in “decision paralysis” which dramatically decreases engagement. Current environments are fragmented and contain a fundamental issue that exists:

- **Prediction without Explanation:** Traditional recommendation systems claim to be proficient in predicting the product a user well like (i.e., collaborative filtering), but do not provide an explanation that is personalized, relevant, and trustworthy in relation to the individual.
- **Explanation without Context:** In contrast, a modern “AI chatbot” may be able to explain complex financial concepts, but lacks deep personalized contextual detail. It is simply not aware of a user’s transaction history, financial goals, nor their risk appetites as historical data, and subsequently produces generic, impersonal advice.

As a result, there is low customer engagement, low financial literacy and a significant loss of opportunity for both consumers as individuals and financial institutions.

2.2 Motivation and Scope

The project goals motivate the development of a single unified structure to fill the gap of a "trustworthy financial assistant" that securely ingests and utilizes granular financial data to give back to users. This system will merge prediction (the Recommender System) with explanation (Generative AI) into one process that users can access in a conversational manner.

- **Scope:** The scope of the project is to deliver a complete, end-to-end, technical proof-of-concept. This proof-of-concept will involve a secure user authentication and management system, a backend data architecture, a working predictive recommender model, and a conversational generative AI interface. The deliverable will be a web application, which is a full stack application that demonstrates this functionality of integration.
- **Limitations:** The recommender model will be trained only with the synthetic financial transaction data generated for this project. The generative AI will leverage the product catalog and user recommendations, but will not be producing real-time market advice or executing trades.

2.3. Project Objectives

There are six primary objectives for the project established in Report 0:

1. **System Design:** Plan an end-to-end system topology for the web-based system (Completed).
2. **Data Synthesis and Analysis:** Create a realistic synthetic dataset of user financial behavior and products (Completed during this Period).
3. **Model Building:** Construct and train a machine-learning recommender model to recommend products by matching user profiles (Next Phase).
4. **AI Application:** Create an Generative AI model (RAG) to support conversational explanations and financial planning applications (Planned).
5. **Interface Development:** Toward building a complete fully functioning full-stack web application for users interaction (Planned).
6. **Evaluation:** Evaluate the overall system performance by reviewing model metrics, conversation quality, and systematic reviews (Planned).

2.4. Foundational Concepts and Literature Review:

Two main foundations of machine learning have underpinned the project's design and are briefly discussed here.

- **Pillar 1: Recommender Systems: The "Prediction" Engine**

Recommender systems in finance have progressed from basic content-based filtering to more potent collaborative filtering (CF) models (5). The most advanced CF techniques frequently leverage Matrix Factorization (MF) (3). A typical strength of MF techniques is managing the sparse nature associated with many financial applications (a user engages only with a small number of the thousands of potential products offered). The premise is to

decompose the large, sparse user-item interaction matrix into lower dimensional "latent factor" matrices. The latent factors represent missing attributes such as a user's "long-term growth" or a product's "low risk." By inferring these latent factors, the model can predict the underlying utility a user would place on unengaged items.

One of the major issues to solve in this domain is the "cold start" problem: a new user has no transaction history and a new product has no interaction data. To address this issue, this project will adopt a Hybrid Recommender System (2) approach where the recommender system will primarily rely on content-based filtering (explicit user data like `income_level` and `risk_tolerance` will be used in generating baseline recommendations) in the early stages with a user with no transaction history and will later shift to a model that incorporates the more robust outcome from Matrix Factorization as the user builds transaction history.

- Pillar 2: Generative AI (The "Explanation" Engine)

The second pillar is Generative AI (in particular, the use of Large Language Models, LLM). It is important to understand that modern LLMs originated from the Transformer concept (11) and a self-attention mechanism. A self-attention mechanism is a means for the model to indicate how much importance it needs to give to certain words in a sentence in determining context. This has particular relevance for finance where the contextual meaning of "risk" or "return" relies heavily on the dialogue occurring around it.

LLMs can, however, "hallucinate" or fabricate facts. This is completely unacceptable in regard to finance. To provide accurate, credible, and personalized recommendations, this project will utilize the approach of Retrieval-Augmented Generation (RAG) (4). RAG is a method that grounds the LLM in a trustworthy and specific knowledge base. In practice, if a user asks a question like, "Why did you recommend this loan?" the RAG will invoke the following three actions:

1. Retrieve: load in the user's profile, the recommender system's output (the `match_score`, for instance, "`match_score: 0.95`"), and the description of the `ProductCatalog` for that specific loan.
2. Augment: load that content to the LLM along with the user's question, which is providing the context (the context is retrieved data),
3. Generate: instruct the LLM to generate an answer using only the context. This provides an explanation that is personalized (i.e., "this loan was recommended because it has a low-interest rate, given your data, which was posted to your low-risk profile...") and accurate.

3. Work Completed

This section describes the specific tasks performed during the "Foundation & Data" phase (Weeks 1-2). This work has a successful completion of the conceptual designs identified in Report 0 into a functional, tangible application skeleton, following the compressed development plan identified by the Professor.

3.1. System Architecture and Tech Stack Setup

The core three-tier system architecture was executed, which included setting up the environments and project structures for each layer.

- Application Layer (Backend)
 - Framework: Flask (Python) was selected as the backend framework, using the Application
 - Pattern (create_app()). This is a particularly important design choice for scalability. Rather than defining a single global Flask app, the app is constructed by a function. This allows:
 - 1) Easy Configuration: The factory function can be passed a configuration (e.g., DevelopmentConfig, TestingConfig), allowing the app to easily switch between a development app.db database (SQLite) and a production connection to a PostgreSQL server.
 - 2) Testability: Multiple iterations of the app can be produced for unit testing.
 - 3) Modularity: It provides a clean way to initialize extensions like Flask-SQLAlchemy and Flask-Login.
 - Modularity (Blueprints): Flask Blueprints organized the application with separate blueprints for auth.py (which handled /login and /register) and main.py (handling core API routes). This helped keep the codebase clean and maintainable, rather than a single giant routes.py file.
- Presentation Layer (Frontend):
 - Framework: A standard React project was initialized via create-react-app.
 - API Communication: A proxy was configured in the React package.json file. This is a development-only feature that resolves the Cross-Origin Resource Sharing (CORS) errors by relaying API requests (e.g., /api/auth/login) from the React server (localhost:3000) to the Flask backend server (localhost:5000). This was tested and confirmed to work.
- Environment and Dependencies:
 - A Python virtual environment (venv) was created to isolate the project's dependencies.
 - All of the python packages (Flask, Flask-SQLAlchemy, Flask-Login, Werkzeug, Faker) were installed and pinned in a requirements.txt file for replicability.

3.2. Database Schema Implementation

The conceptual database schema (Report 0, page 6) was coded with Flask-SQLAlchemy (1). This Relational Mapper (ORM) allows you to define your database tables as Python classes, speeding up development and minimizing the potential for errors from raw SQL code.

The following five models were coded in app/models.py:

1. **User Model:** This model inherits from UserMixin for easy integration with Flask-Login and stores all user-specific data.
 - id: Primary key.
 - username: A string, indexed and unique to log in.
 - password_hash: A string to store a hashed password with 256 characters.
 - income_level & risk_tolerance: Strings (e.g., 'low', 'medium', 'high') which are primary features for the content-based (cold start) recommender.
2. **ProductCatalog Model:** This defines the "items" which can be recommended.
 - id: Primary key.
 - name: The name of the product (e.g., "Global Tech ETF").
 - description: A text field to contain the official product details that is the primary text knowledge source for the RAG system.
 - category, interest_rate, etc. Other metadata to filter on.
3. **Transaction Model:** This is the "interact" table which ties users and products together.
 - id: Primary key.
 - user_id: A foreign key linked to User.id.
 - product_id: A foreign key linked to ProductCatalog.id.
 - transaction_type, amount, and timestamp: Metadata about the interact.

The models provided and their relationships (e.to-many) are utilized to construct the user-item interaction matrix. After they were defined, the flask db init, flask db migrate, and flask db upgrade commands were executed to create the initial app.db (SQLite) file with the correct schema.

3.3. Basic User Authentication Implementation

The core security layer has been implemented in full.

- **Password Hashing:** The User model now includes set_password(password) and check_password(password) methods. The set_password method uses Werkzeug.security.generate_password_hash in order to convert a plain-text password into a securely-hashed password, including a salt to mitigate rainbow-table attacks.
- **Session Management:** Flask-Login (12) has been initialized and configured. It handles the entire user session lifecycle, starting with logging in the user (secure session cookie created) and logging out the user (session company destroyed). A

user_loader callback has also been implemented allowing Flask-Login to pull the current user object from the database using the session cookie.

- **API Endpoints:** Two API endpoints were developed in the auth blueprint, /register and /login.
 - /register accepts a JSON payload, verifies that the username is available, generates a new User object, sets the password to a hashed version of the received password, and then adds it to the database.
 - /login accepts a JSON payload, identifies the user, and validates the provided password using the function check_password(). If validation is successful, the Flask-Login function login_user() is called to create the session.

3.4. Synthetic Dataset Generation

To fill the database, a script, seed_data.py, was created. This was a vital step in the process because the effectiveness of the recommender system relies entirely on the quality of the training data.

- **Methodology:** A logical and tiered approach to generation was employed.
 1. Products: Step one was to create 50 ProductCatalog items and save them (for example, "High-Yield Savings," "Student Loan," "Global Tech ETF").
 2. Step two was to create 1,000 User profiles using Faker (10). Each user is randomly assigned an income_level and risk_tolerance.
 3. Step three was the most difficult. 1,000,000 transactions were generated. To create plausible behavioral patterns for the ML model to find, logic was employed statistically:
 - Users classified as risk_tolerance='high' and income_level='high' had 10 times the likelihood of being aligned with "investment" or "ETF" products.
 - Users that were risk_tolerance='low' were mostly associated with "savings" or "loan_payment" products.
 - This creates a densely populated and notably non-random relationship between user features and user interactions, which is a necessity for the hybrid recommender to learn.
- **Results:** The app.db file has now been filled with a large, complex, realistic dataset, completing this objective for the "Data Synthesis" task.

4. Preliminary Implementation

This section provides the concrete code artifacts and implementation details for the work completed.

```
# app/models.py
# This is a simplified representation of the implemented models.
```

```

from . import db # db object from Application Factory
from flask_login import UserMixin
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime, timezone

class User(UserMixin, db.Model):
    __tablename__ = 'user'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), index=True, unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)

    # Content-based features for cold-start
    income_level = db.Column(db.String(64), index=True)
    risk_tolerance = db.Column(db.String(64), index=True)

    # Relationship to transactions
    transactions = db.relationship("Transaction", backref='owner', lazy='dynamic')

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

class ProductCatalog(db.Model):
    __tablename__ = 'product_catalog'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), index=True)
    description = db.Column(db.Text)
    category = db.Column(db.String(64), index=True)
    interest_rate = db.Column(db.Float)

    # Relationship to transactions
    transactions = db.relationship("Transaction", backref='product', lazy='dynamic')

class Transaction(db.Model):
    __tablename__ = 'transaction'
    id = db.Column(db.Integer, primary_key=True)
    amount = db.Column(db.Float)
    transaction_type = db.Column(db.String(64), index=True)
    timestamp = db.Column(db.DateTime, index=True, default=lambda:
datetime.now(timezone.utc))

    # Foreign keys establishing the relationships
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    product_id = db.Column(db.Integer, db.ForeignKey('product_catalog.id'))

```

4.2. Implemented Authentication API (Code)

The following code from app/auth.py shows the functional endpoints for user registration and login.

```
# app/auth.py
# This is a simplified representation of the authentication routes.

from flask import Blueprint, request, jsonify
from .models import User
from . import db
from flask_login import login_user, logout_user, current_user

auth_bp = Blueprint('auth', __name__, url_prefix='/api/auth')

@auth_bp.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    if not data or not 'username' in data or not 'password' in data:
        return jsonify({'error': 'Missing data'}), 400

    if User.query.filter_by(username=data['username']).first():
        return jsonify({'error': 'Username already exists'}), 400

    new_user = User(
        username=data['username'],
        income_level=data.get('income_level', 'medium'),
        risk_tolerance=data.get('risk_tolerance', 'medium')
    )
    new_user.set_password(data['password'])
    db.session.add(new_user)
    db.session.commit()

    return jsonify({'message': 'User registered successfully'}), 201

@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    if not data or not 'username' in data or not 'password' in data:
        return jsonify({'error': 'Missing data'}), 400

    user = User.query.filter_by(username=data['username']).first()

    if user is None or not user.check_password(data['password']):
        return jsonify({'error': 'Invalid username or password'}), 401

    login_user(user, remember=True)
    return jsonify({'message': 'Login successful'}), 200
```

```
@auth_bp.route('/logout', methods=['POST'])
def logout():
    logout_user()
    return jsonify({'message': 'Logout successful'}), 200
```

4.3. Challenges and Resolutions

There were no significant blockers identified, but there were a few scope-defining decisions and minor Challenges.

- Challenges (Decision): Database Option (SQLite vs Production DB).
- Chosen Solution: SQLite was selected intentionally for this foundational stage of the project (Weeks 1-2) because it has zero configuration to get started, operates as a single file (app.db), and is optimal for fast prototyping and schema changes. Using the Flask-SQLAlchemy ORM abstracts the database completely, meaning that the application could be migrated to a production-grade database such as PostgreSQL with a simple string change in the configuration file, without needing to touch the Python model code at all.
- Challenge (Process): API Endpoint Testing.
- Chosen Solution: It is not possible to test the backend API routes from the browser (which only allows GET requests) so the Postman app was used for all API testing. A Postman "collection" was built specifically to test the authentication flow.
 1. Then, a POST request was sent to /api/auth/register with a JSON body of {"username": "testuser", "password": "123"}.
 2. Confirmed a 201 Created response.
 3. Then, a POST request was sent to /api/auth/login with the same credentials.
 4. Confirmed the 200 OK response and the session cookie in the response headers.
 5. Lastly, a POST request to /api/auth/login was sent with an invalid password.
 6. Confirmed the 401 Unauthorized error response.

This confirmed that the entire authentication and hashing logic all work properly.

- Challenge (Implementation): Circular Imports with the Application Factory.
- Resolution: A common problem with the factory pattern is circular imports (e.g., app.py imports models.py, but models.py must import the db object from app.py). This was resolved by instantiating the extensions (e.g., db = SQLAlchemy()) in a separate extensions.py file which does not require an app. Then, in the create_app() factory, you simply pass the app to db.init_app(app) to finalize the binding.

5. Next Steps

The project is on schedule. We have completed the "Foundation & Data" phase, so we are now entering the "Model Building & Integration" phase (Weeks 3-6). Our planned tasks for the next two-week reporting period will include:

1. Development of Recommender Module (Weeks 3-4):
 - Data Preprocessing: Create a script that will read the Transaction data from the SQLite database into a Pandas DataFrame.
 - Pivot the Data: Organize the DataFrame into the user-item interaction matrix that we will need.
 - Model Training: Use Scikit-learn's TruncatedSVD (a variant of Matrix Factorization) on the interaction matrix to create the latent factor models.
 - Hybrid Logic: Develop the "cold start" logic. If the user has less than 5 transactions, the recommender system will apply a content-based filter (such as matching risk_tolerance to product.category). If the user has more than 5 transactions, it will apply the SVD model.
2. Conversational Module (Chatbot MVP) (Week 5):
 - Frontend: Build out the basic chatbot UI component in React. This will be a very basic chat window with text input.
 - State Management: The React component will manage the conversation state (the list of messages) and make API calls to the backend.
3. LangChain Pipeline (RAG) (Week 6):
 - Integrating LLM: Build out the backend RAG pipeline with the LangChain library.
 - Retrieving: The pipeline will be set up to retrieve from the ProductCatalog table in response to the question posed by the user.
 - API Endpoint: A new endpoint for Flask API (e.g. /api/chat) will be created to accept messages from the user in the React frontend and send them to the LangChain pipeline.
4. Preliminary End-to-End Testing (Week 6):
 - Conduct the initial end-to-end data flow test.
 1. User logs in (React).
 2. System generates recommendations (Flask + SVD).
 3. User asks about a recommendation (React Chat).
 4. System generates a RAG-based answer (Flask + LangChain).

6. References

This section includes academic and technical references for the core concepts and technologies discussed and implemented.

1. Bayer, M. (2025). *SQLAlchemy: The Python SQL toolkit and Object Relational Mapper*. Retrieved from <https://www.sqlalchemy.org/>
2. Burke, R. Hybrid Recommender Systems: Survey and Experiments. *User Model User-Adap Inter* **12**, 331–370 (2002). <https://doi.org/10.1023/A:1021240730564>.
3. Koren, Y., Bell, R.M., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*, 42.

4. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
5. Linden, Greg & Smith, B. & York, J.. (2003). Linden G, Smith B and York J: 'Amazon.com recommendations: item-to-item collaborative filtering', Internet Comput. IEEE, , 7. Internet Computing, IEEE. 7. 76 - 80.1109/MIC.2003.1167344.
6. Meta. (2025). *React: The library for web and native user interfaces*. Retrieved from <https://react.dev/>
7. Pallets Projects. (2025). *Flask: The Python micro framework for building web applications*. Retrieved from <https://flask.palletsprojects.com/>
8. Pallets Projects. (2025). *Werkzeug: The comprehensive WSGI web application library*. Retrieved from <https://werkzeug.palletsprojects.com/>
9. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(Oct), 2825-2830.
10. Tippmann, F. (2025). *Faker: A Python package that generates fake data*. Retrieved from <https://faker.readthedocs.io/>
11. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
12. Zelenin, M. (2025). *Flask-Login: User session management for Flask*. Retrieved from <https://flask-login.readthedocs.io/>